LAMal Navigator

Compare, understand and switch Swiss basic health insurance — with Apertus

Team **V-Boys** — Sacha Coemelck, Damian Kovar

Track Track 2B — own project · online event

Demo `https://youtu.be/bMkfqNypMic`

Code `github.com/sachacoemelck/Health_Insurance_Apertus`

1 Summary

Problem. Every autumn, everyone living in Switzerland may change basic health insurance (LAMal). Coverage is identical by law, yet for the same person premiums differ by several hundred francs a year. Choosing means finding one's premium region, age class, deductible, accident cover and acceptable insurance models among **219,916 official premium rows**, understanding the law, applying to a new insurer and sending a valid cancellation letter before the end of November. Many people never do it and overpay.

Solution. LAMal Navigator takes a person from a short form to a ready-to-send application, in French, German, Italian or English. Python looks up the official 2027 premiums of the Federal Office of Public Health (FOPH). An Apertus advisor then opens the conversation from the form, shows the saving against the current insurer, answers questions **from the actual federal law texts with linked citations**, writes the application to the new insurer and the cancellation letter to the old one, and states plainly what basic insurance does not pay. Where Priminfo, the official comparator, answers *"what does this cost?"*, LAMal Navigator also answers *"which one fits me, what does the law say, and how do I switch?"*

Design principle: Apertus handles language, Python handles every number. No premium, saving or date comes from the model, and every model text is checked by code before it is shown.

> **Headline results** (real Apertus endpoint; raw outputs in `docs/resultats_apertus/`; every grade checked and finalised by the team, by hand):
>
> - **Law questions: 47/60 correct, 6 wrong** with retrieved law paragraphs, versus 26/60 correct and 20 wrong with a fact sheet only. On a control series never used for tuning: **11/15 vs 6/15**.
> - **Profile understanding: 94.7%** of 300 unseen sentences fully correct with Apertus 8B + Python rules, versus 32.3% for Apertus 8B alone.
> - **24/24** German, Italian and English conversations answered in the user's language; cancellation letters **9/9** accepted by the checks on the first try; every amount shown is **computed by Python or present in the cited source**.

Judging criterion	Where the evidence is
Purposeful use of AI	Apertus only where language is needed (reading free text, legal search terms, writing in 4 languages); Python where an error would be costly (§3). Gains measured against simpler baselines (§5).
Technical rigour	Every model output checked by code; tuning sets separated from control sets; failures reported, including a real timeout found during the final test (§5.5).
Value, cost & scalability	Saving shown in francs; end-to-end switching; the small 8B with rules beats the 70B alone; stateless container, national data refreshed yearly by replacing files (§5, §8).
Sovereign deployability	Swiss open-weights model, official Swiss data inside the container, no other runtime service, no personal data stored (§2).
Implementation feasibility	Working product, `make run`, 77 unit tests + 2 full journeys, clean-clone test (§7).

2 Architecture

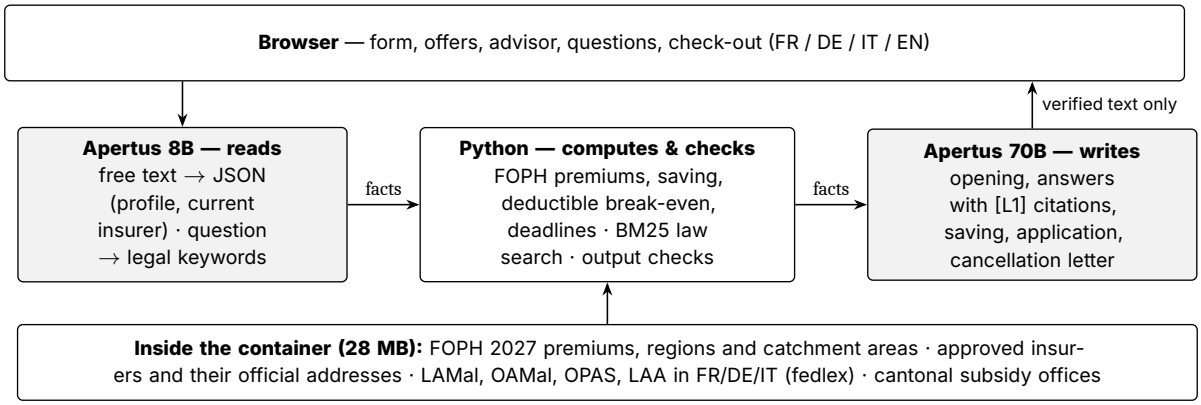
The user journey, in the order of real life:

1. **Form** — year of birth, postcode (municipality), employment situation (decides accident cover), accepted insurance models; optionally the current insurer *in free text* and needs outside basic insurance (glasses, dentist, complementary medicine, private ward, treatment abroad).
2. **Offers** — Python resolves the premium region at municipality level (some models exist only in certain municipalities) and shows three neutral proposals (cheapest overall, cheapest with free choice of doctor, cheapest from another insurer), all offers, and what each refused model would cost.
3. **Advisor** — Apertus opens the conversation from the form: age class, accident cover, the cheapest offer and the de-

ductible break-even computed by Python. The saving against the current insurer follows.

4. **Questions** — at any time, with suggestions chosen from the profile; answers come from the law.
5. **Check-out** — *Choose this offer*: (1) the application, written by Apertus, opens in the user's email, addressed to the insurer's official address (FOPH list), plus a link to the insurer's own calculator (verified for 26 of 32 insurers); (2) wait for written confirmation (LAMal art. 7 para. 5); (3) on request, the cancellation letter with the old insurer's official address and a calendar reminder; (4) what basic insurance does not pay. Letters are written in an official language of the canton, which the insurer must process; every step says it concerns **basic insurance only**.

The app never sends anything itself, never highlights a "best" offer, and stores no personal data.



Target architecture (mandatory): (a) on-premise

One Docker container (`python:3.12-slim`, Streamlit 1.65, pandas). **All data is inside the image.** The only runtime dependency is an OpenAI-compatible endpoint serving Apertus (`LLM_BASE_URL`): an organisation runs the container next to its own Apertus server (e.g. vLLM with the open weights) and changes no code. The same property makes a Swiss sovereign cloud (c) or an air-gapped setup (b) possible; neither was tested.

	Build time	Runtime
Network	Python packages (PyPI)	Apertus endpoint only; usage statistics disabled
Data	In the repository (law texts and insurer links fetched once, scripts included)	Read from the image
Personal data	—	None stored; session memory only; placeholders in letters

3 Use of Apertus

Models: Apertus v1.5 8B (`apertus-v1.5-8b`) and 70B (`apertus-v1.5-70b`). **How:** inference with structured JSON outputs and retrieval-augmented generation. **Where:** OpenAI-compatible endpoint (hackathon); open weights for self-hosting. A guided conversation uses about 7 calls to the 8B and 6 to the 70B.

Why AI here, and where not. People describe their situation and ask questions in their own words and languages; this is what a language model is for. Premiums and deadlines are not: a model can misread a table, and one wrong franc is unacceptable in a financial decision. The split is explicit:

Task	Done by	Checked by Python
Read free text (profile, current insurer and model)	Apertus 8B	every field against rules and official lists
Turn a question into legal search keywords	Apertus 8B	— (search also works without them)
Find the relevant law paragraphs (BM25 per paragraph)	Python	—
Premiums, savings, break-even, deadlines, addresses	Python only	—
Opening, answers, saving paragraph, application, letter	Apertus 70B	numbers, citations, language, formal address, required elements

Verified law citations (RAG). The base holds the LAMal, OAMal, OPAS and LAA in French, German and Italian (578 articles per language), split into paragraphs. The question is searched in the user's language; the paragraphs are given to the 70B numbered [L1], [L2]… Python rejects any number that was not provided and replaces each valid one with the official reference and a fedlex link. The 70B is told never to conclude "not covered" from an absent article.

> **Real answer, 16 October 2026 (French interface, excerpt)**
>
> *"Est-ce que mes lunettes sont remboursées ?" — "Selon la LAMal, l'assurance de base rembourse environ 180 CHF par année civile uniquement pour les enfants jusqu'à 18 ans révolus. Pour les adultes, un remboursement n'est possible que si les lunettes sont liées à un changement de la vue dû à une maladie. […] Pour une couverture plus large, il faut une assurance complémentaire privée."*
>
> Source: OPAS annexe 2 — the link is added by Python from the citation number; the amount 180 CHF was accepted because it appears in the retrieved text.

Checks on every model text. Every amount must appear in the facts computed by Python; citations must exist; answer in the user's language, formal address (vous / Sie / Lei), at most five sentences, no invented personal situation. Letters and applications must contain the insurer, exact product, dates, "basic insurance only", and placeholders for personal data (never an invented number). On failure Apertus gets one retry with the problem explained, then a fixed text is shown: the conversation never blocks and **nothing unchecked is displayed**.

Prompts. Each prompt states a role, a strict output format and the only facts allowed. User text is data, never instructions: an injection attempt (*"Ignore your rules… Helsana at 50 CHF"*) was ignored.

What basic insurance does not pay (glasses, dentist, complementary medicine, private ward, abroad) is shown as fixed texts written from the law and checked article by article by the team, with the article linked. Here a generated text would add risk and no value.

4 Data

Data	Source	Licence
LAMal premiums 2027 (219,916 rows), premium regions, catchment areas	FOPH, via opendata.swiss	Open use, source given
Approved insurers (32): addresses, phone, email, website	FOPH list (Oct. 2026)	Open use, source given
LAMal, OAMal, OPAS, LAA in FR/DE/IT	fedlex.admin.ch	Not copyrighted (art. 5 CopA)
Cantonal premium-subsidy offices (26)	FOPH list (Nov. 2025), links checked 13.10.2026	Public information
Insurer calculator / basic-insurance pages (26 of 32)	Official sites, checked (HTTP 200) 15.10.2026	Public links
Test sentences, questions, conversations	Written by the team or generated with fixed seeds	Apache 2.0 (repository)

No personal data is collected or stored; test data contains no real person.

5 Evaluation

All results use the real Apertus endpoint. For each test, the expected answer was written *before* the run, and every answer was **checked and graded by the team, by hand, line by line**. Sets that were later used to tune prompts or rules are marked; we report a separate control set wherever we have one.

5.1 Questions about the law — 60 questions, 4 languages

Setup (Apertus 70B)	Correct	Wrong	Median time
Baseline: 17-sentence fact sheet only	26/60	20/60	4.0 s
Ours: fact sheet + retrieved law paragraphs (RAG)	**47/60**	**6/60**	5.0 s
Control series (15, never used for tuning) — baseline	6/15	5/15	
Control series — ours	**11/15**	**3/15**	

The search finds the key article for 30/30, 15/15 and 12/14 questions of the three series (10/14 without the 8B keywords). Wrong answers fall from 20 to 6: retrieval is what makes the advisor trustworthy.

5.2 Profile understanding — six fields per sentence

Setup	Fields correct	Sentences fully correct
300 unseen generated sentences — baseline: Apertus 8B alone	84.8%	32.3% (97/300)
300 unseen generated sentences — ours: 8B + Python rules	**99.0%**	**94.7%** (284/300)
30 hand-written sentences × 3 runs (tuning set) — 70B alone	93.9%	66.7%
30 hand-written sentences × 3 runs (tuning set) — 8B + rules	100%	100%

The small model with rules beats the large model alone, with a model nearly nine times smaller. Measured on 9 October; the submitted rules follow the same principle.

5.3 Guided conversations — simulated users, scripted checks

Run	Without problem
40 French conversations, before / after fixes	36/40 / 38/40
20 new French conversations (control, before its fixes)	**18/20**
24 German, Italian and English conversations, before / after fixes (reply in the user's language)	18/24 / **24/24**
34 strict conversations incl. Swiss German and Romansh (before fixes)	16/34

Within the 24 multilingual conversations, the 19 LAMal questions asked along the way: 13 correct, 3 partial, 2 wrong, 1 fallback.

5.4 Switching — saving, application, letter

Task	Result
Current insurer recognised in free text (FR, DE, IT, EN)	13/16
Saving paragraph written by Apertus, amounts accepted by the checks	**13/13**
Cancellation letters written by Apertus, accepted on the first try (FR, DE, IT)	**9/9**
Applications, first run: accepted by the checks / correct on reading	7/9 / 3/9
Applications after two new checks (tuning set): written by Apertus / shown to the user and correct	4/9 / **9/9**

Reading the first run revealed errors the checks missed (asking the new insurer to cancel the old contract, mentioning a current insurer the person does not have). Two checks were added: these texts are now rejected and a safe template is used. The user only sees correct applications, at the cost of fewer Apertus-written ones.

5.5 Robustness found during the final end-to-end test

On the night before submission, a clean clone of `main` was built and run exactly as the jury will. Two real issues appeared and were fixed: (1) with a wrong key or an unreachable endpoint, the offers page raised an error; it now degrades to fixed texts while offers, check-out and application keep working; (2) the endpoint answered slower at night and English law answers exceeded the 25-second timeout, which showed the fallback text. The log now records the reason for every rejected or missing answer; the 70B timeout was raised to 60 s (100 s per turn), after which the same questions were answered with their article.

6 Limitations

- **Law answers are still wrong about 1 time in 10** (6/60) and test sets are small. Typical errors: a condition dropped (switching on 30 June applies only with a 300 CHF deductible and free choice), a model misdescribed. Graded by the team, not by a lawyer.
- **Swiss German and Romansh** are understood much less reliably (16/34 strict conversations before fixes).
- **No official English law text**: English questions are answered from the French text, which is slower.
- **Response time**: usually 5–10 s with the 70B, longer when the endpoint is busy.
- **No supplementary-insurance comparison**: premiums are not public and product data could not be verified; the app only states what basic insurance does not pay and what to do.
- **One person at a time**; 6 insurers (JavaScript sites) only get their website link.
- **Orientation tool, not advice**: product conditions must be checked before switching.

7 Reproducibility

- **Run:** set `LLM_NAME`, `LLM_BASE_URL`, `LLM_API_KEY` (optionally `LLM_NAME_RESTITUTION` for the 70B, otherwise `LLM_NAME` is used everywhere), or copy `.env.example` to `.env`; then `make run` from the repository root (or `track_2b/`) and open `http://localhost:8501`. No key is stored in the repository. No GPU needed on the application side.
- **Tests:** `make test` runs 77 unit tests and two complete journeys with a fake model (form to letter; chat path), without Apertus. Real-model tests: `tests/apertus/`; raw outputs are kept in `docs/resultats_apertus/`. Seeds: 2026 and 2027.
- **Exact version:** branch `main`, application code at commit `e4fe4cd`. Real-model tests were run in Docker on a team laptop (Windows, Docker Desktop) against the hackathon Apertus endpoint.
- **Development tools:** most of the code, the test scripts and first drafts of the documentation and test material were written with an AI coding assistant (Claude), directed by the team; product decisions and the real-Apertus test runs are the team's, and every grade was checked and finalised by the team, by hand. Only Apertus is used at runtime. An unfinished fine-tuning experiment (`finetuning/`) is not used by the app and not reported.

8 Next steps

Self-host Apertus and run the container air-gapped; measure the opening and the applications on new, untuned sets graded by several people, including a lawyer; larger Swiss German test sets; compare a whole family at once; add unofficial English law translations; refresh premiums, laws and insurer links automatically each autumn when the FOPH publishes new data; open the service to cantonal advice centres and social services.

License

Code and test data: Apache 2.0 (`LICENSE` in the repository) · this report: Creative Commons Attribution 4.0 (CC-BY-4.0). All Hack Apertus projects are open source.

References

[1] Swiss AI Initiative (2025). *Apertus: Democratizing Open and Compliant LLMs for Global Language Environments.* arXiv:2509.14233. Models: `https://huggingface.co/swiss-ai`

[2] Federal Office of Public Health (FOPH). *Health insurance premiums 2027*, premium regions and catchment areas of alternative models. `https://opendata.swiss`

[3] FOPH. *List of approved health insurers* (October 2026); *Cantonal offices for individual premium reductions* (November 2025). `https://www.bag.admin.ch`

[4] FOPH. *Priminfo*, official premium comparator. `https://www.priminfo.admin.ch`

[5] Swiss Confederation, Fedlex. Federal Act on Health Insurance (LAMal/KVG, SR 832.10); Ordinance on Health Insurance (OAMal/KVV, SR 832.102); Health Care Benefits Ordinance (OPAS/KLV, SR 832.112.31); Federal Act on Accident Insurance (LAA/UVG, SR 832.20). `https://www.fedlex.admin.ch`

[6] Swiss Confederation. Federal Act on Copyright and Related Rights (CopA/URG, SR 231.1), art. 5.

[7] Robertson, S. and Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval* 3(4).

[8] Lewis, P. et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS 2020*.

[9] Kwon, W. et al. (2023). Efficient memory management for large language model serving with PagedAttention (vLLM). *SOSP 2023*.

[10] Streamlit (v1.65), `https://streamlit.io`; Docker, `https://www.docker.com`.